

Jira Daily Bug Reporter

Step-by-Step Implementation Guide for a Linux VM

Jira Cloud, Gmail SMTP Relay, Google Chat, Python Standard Library

This guide explains how to build a corporate-friendly Jira bug reporter that pulls issues from Celestica Jira Cloud, generates an HTML report like the screenshots, sends it by email, and optionally posts a short Google Chat summary. It assumes you are a beginner, but it keeps the design production-shaped.

Contents

1	What You Are Building	4
1.1	Target output	4
1.2	Main design decision	4
2	Architecture	4
2.1	High-level flow	4
2.2	Components	5
2.3	Why this architecture works in a corporate network	5
3	Official References Used	5
4	Information You Need Before Coding	5
5	Phase 0: VM Pre-flight Checks	6
5.1	Check Python	6
5.2	Check outbound Jira access	6
5.3	Check if the VM uses a proxy	6
6	Phase 1: Jira Authentication Test	7
6.1	Create an Atlassian API token	7
6.2	Export temporary test variables	7
6.3	Test whoami	7
6.4	Common failures	7
7	Phase 2: Discover Project and Fields	8
7.1	List visible projects	8
7.2	Test JQL search	8
7.3	Discover status names	8
7.4	Discover custom fields	8
8	Phase 3: Email Path Validation	9
8.1	Test network access to SMTP relay	9
8.2	Ask IT this exact question	9
8.3	Fallback email options	9
9	Phase 4: Google Chat Validation	9
9.1	Create a webhook	9
9.2	Test the webhook	10
10	Phase 5: Create the Project Skeleton	10
10.1	Create folders	10
10.2	Recommended folder structure	10
11	Phase 6: Configuration Design	11
11.1	Example config	11
11.2	Secrets through environment variables	11

12 Phase 7: Jira Client Design	11
12.1 Jira API behavior	11
12.2 Pagination model	12
12.3 Data you should normalize	12
13 Phase 8: Categorization Rules	12
13.1 Core sections	12
13.2 Age calculation	13
13.3 Age color	13
14 Phase 9: Analytics Rules	13
14.1 Required metrics	13
14.2 Email-safe chart strategy	13
15 Phase 10: HTML Email Design	14
15.1 Email rendering rules	14
15.2 Recommended email layout	14
15.3 Columns to include	15
16 Phase 11: Email Sender Design	15
16.1 SMTP configuration	15
16.2 Python standard library modules	15
16.3 Failure handling	15
17 Phase 12: Google Chat Sender Design	16
18 Phase 13: Testing Plan	16
18.1 Beginner-friendly rule	16
18.2 Test order	16
18.3 Useful commands	16
19 Phase 14: Logging and Output	17
19.1 Recommended output files	17
19.2 Log what matters	17
20 Phase 15: Deployment With Systemd	17
20.1 Create install directory	17
20.2 Environment file	17
20.3 Service file	18
20.4 Timer file	18
20.5 Enable and test	18
21 Phase 16: Security Checklist	19
22 Phase 17: Troubleshooting	19
23 Implementation Milestones	19
23.1 Milestone 1: API proof	19
23.2 Milestone 2: Local HTML report	20
23.3 Milestone 3: Email report	20
23.4 Milestone 4: Production schedule	20

24 Version Roadmap	20
24.1 Version 1: Minimum useful production tool	20
24.2 Version 2: Better reporting	20
24.3 Version 3: Team-scale tool	21
25 Recommended First Week Plan	21
26 Definition of Done	21
27 Appendix A: Minimal Dry-run Command Set	21
28 Appendix B: IT Approval Checklist	22
29 Appendix C: What To Build First	22

1 What You Are Building

You are building a scheduled reporting tool for Jira bugs. The tool runs on a Linux VM, authenticates to Jira Cloud, fetches issues with JQL, groups them into useful sections, renders a clean HTML email, and sends it through the approved company email path.

1.1 Target output

The final email should include:

- Greeting and report title.
- Overall summary counts.
- Counts by status and priority.
- Optional visual distribution using email-safe HTML bars.
- All Bugs table.
- Feedback Needed Bugs table.
- Test / Verify Fix Bugs table.
- Newly Created Bugs table for the last 2 days.
- Footer with automation owner/contact.

1.2 Main design decision

For version 1, keep the tool intentionally simple:

- No web server.
- No dashboard.
- No browser scraping.
- No database server.
- No Docker required.
- No large third-party Python framework.

The first production version can be one scheduled Python job using mostly Python's standard library.

2 Architecture

2.1 High-level flow

```
cron/systemd timer
-> Python report job
  -> Jira REST API
  -> normalize issue data
  -> categorize bugs
  -> compute summary metrics
  -> render HTML email
  -> save latest_report.html
  -> send via Gmail SMTP relay
  -> optional Google Chat notification
  -> write logs
```

2.2 Components

Component	Responsibility
Scheduler	Runs the report daily or weekly using cron or systemd timer.
Jira Client	Calls Jira Cloud REST API using API token authentication.
Normalizer	Converts Jira JSON into a simple internal issue object.
Categorizer	Splits issues into All, Feedback, Test / Verify Fix, and New bugs.
Analytics	Counts by status, priority, assignee, reporter, and aging.
Renderer	Builds a Gmail-safe HTML report with inline CSS.
Email Sender	Sends HTML email through the approved SMTP relay.
Chat Sender	Optionally posts a short summary to a Google Chat webhook.
Storage	Saves snapshots, latest report, and logs locally. SQLite is optional for version 1.

2.3 Why this architecture works in a corporate network

- It uses outbound HTTPS to Jira Cloud.
- It uses the company's approved outbound email route.
- It does not need inbound firewall rules.
- It does not expose a port.
- It can run under a service account on a locked-down Linux VM.
- It has a readable failure path: logs plus latest generated HTML.

3 Official References Used

Use these as the source of truth when company infrastructure behaves differently:

- [Jira Cloud REST API v3 introduction](#)
- [Jira basic auth with email and API token](#)
- [Jira JQL search API support article](#)
- [Google Workspace SMTP relay](#)
- [Google Chat incoming webhooks](#)

Note: As of this guide, Jira Cloud's recommended JQL search endpoint is `/rest/api/3/search/jql`. Atlassian's support article says the older `/rest/api/3/search` endpoint is legacy/removed for Jira Cloud.

4 Information You Need Before Coding

Collect this before writing the script:

Item	Example / Why it matters
Jira base URL	<code>https://celestica.atlassian.net</code>
Project key	Example: RBC, CBS, or another key used by your team.
Report name	Example: CruiseBay RBC Daily Bug Report.
Jira account	Prefer a service account if company policy allows it.
API token	Required for basic auth scripts against Jira Cloud.
JQL query	Defines exactly which bugs belong in the report.
Status names	Example: Feedback, Test, Fixed, Open.
Priority names	Example: P1, P2.
Email sender	Example: <code>bug-report@celestica.com</code> .
Email recipients	To/Cc list for the report.
SMTP route	Google SMTP relay or internal company SMTP relay.
Chat webhook	Optional. Only needed if Google Chat summary is wanted.

5 Phase 0: VM Pre-flight Checks

Run these commands on the Linux VM.

5.1 Check Python

```
python3 --version
python3 -m venv --help >/dev/null && echo "venv available"
```

You want Python 3.8 or newer. Python 3.10+ is better.

5.2 Check outbound Jira access

```
curl -I https://celestica.atlassian.net
```

Expected result:

- HTTP response headers appear.
- It does not hang.
- It does not show DNS failure, TLS failure, or proxy block.

5.3 Check if the VM uses a proxy

```
env | grep -i proxy
```

If you see proxy variables, save them. The Python script may need to run under the same environment.

Checkpoint: Do not start coding until the VM can reach `https://celestica.atlassian.net`. If this fails, ask IT whether the VM has outbound HTTPS access to Atlassian Cloud.

6 Phase 1: Jira Authentication Test

6.1 Create an Atlassian API token

Go to:

```
https://id.atlassian.com/manage-profile/security/api-tokens
```

Create a token named something like:

```
jira-daily-bug-reporter-linux-vm
```

Risk: Do not paste the API token into source code, GitHub, screenshots, or chat messages. Keep it in environment variables or a protected systemd environment file.

6.2 Export temporary test variables

```
export JIRA_BASE="https://celestica.atlassian.net"
export JIRA_EMAIL="your.email@celestica.com"
read -s JIRA_API_TOKEN
export JIRA_API_TOKEN
```

6.3 Test whoami

```
curl -sS \
-u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
-H "Accept: application/json" \
"$JIRA_BASE/rest/api/3/myself" | python3 -m json.tool
```

Expected result:

- JSON output with account details.
- No 401 Unauthorized.
- No HTML login page.

6.4 Common failures

Failure	Meaning / Fix
401 Unauthorized	Email/token is wrong, token was copied incorrectly, or account cannot use API token.
403 Forbidden	Account authenticates but lacks permission for the resource.
HTML login page	Wrong URL, proxy issue, SSO interception, or using browser flow instead of REST auth.
TLS error	VM lacks proper CA certificates or company proxy is intercepting TLS.

Checkpoint: Continue only after `/rest/api/3/myself` returns valid JSON.

7 Phase 2: Discover Project and Fields

7.1 List visible projects

```
curl -sS \  
-u "$JIRA_EMAIL:$JIRA_API_TOKEN" \  
-H "Accept: application/json" \  
"$JIRA_BASE/rest/api/3/project/search?maxResults=50" | python3 -m json.tool
```

Find the project key for the bugs you need to report.

7.2 Test JQL search

Set your project key:

```
export PROJECT_KEY="RBC"
```

Try the JQL endpoint:

```
curl -sS -G \  
-u "$JIRA_EMAIL:$JIRA_API_TOKEN" \  
-H "Accept: application/json" \  
--data-urlencode "jql=project = $PROJECT_KEY ORDER BY created DESC" \  
--data-urlencode "maxResults=5" \  
--data-urlencode "fields=key,summary,status,priority,assignee,reporter,created,updated" \  
"$JIRA_BASE/rest/api/3/search/jql" | python3 -m json.tool
```

Expected result:

- JSON with an `issues` array.
- Each issue has `key` and `fields`.

7.3 Discover status names

```
curl -sS \  
-u "$JIRA_EMAIL:$JIRA_API_TOKEN" \  
-H "Accept: application/json" \  
"$JIRA_BASE/rest/api/3/status" | python3 -m json.tool
```

Write down the exact status names used by your board. Spelling matters.

7.4 Discover custom fields

```
curl -sS \  
-u "$JIRA_EMAIL:$JIRA_API_TOKEN" \  
-H "Accept: application/json" \  
"$JIRA_BASE/rest/api/3/field" | python3 -m json.tool > jira_fields.json  
  
grep -i "reason" jira_fields.json  
grep -i "feedback" jira_fields.json  
grep -i "customer" jira_fields.json
```

This is useful if the report needs a column like `Reason`.

Checkpoint: By the end of Phase 2, you should know project key, status names, priority names, and any custom field IDs needed for the report.

8 Phase 3: Email Path Validation

Because your company uses Gmail/Google Workspace, the cleanest route is usually Google Workspace SMTP relay. However, only your IT/admin team can confirm whether your VM is allowed to send through it.

8.1 Test network access to SMTP relay

```
nc -vz smtp-relay.gmail.com 587
```

If nc is unavailable:

```
openssl s_client -starttls smtp -connect smtp-relay.gmail.com:587
```

8.2 Ask IT this exact question

Can this Linux VM send automated report emails through smtp-relay.gmail.com:587?
 If yes, should the VM public IP be allowlisted, or should we use SMTP authentication?
 What From address should the automation use?
 Are there recipient or daily sending limits for this report?

8.3 Fallback email options

Option	When to use
Google SMTP relay	Best if Workspace admin approves VM or sender.
Internal SMTP relay	Best if company has an existing mail relay for servers.
Gmail app password	Usually not ideal in corporate SSO environments. Use only if approved.
Manual HTML output	Temporary fallback while waiting for IT approval.

Checkpoint: Before implementing email sending, confirm which SMTP host, port, sender, and auth method are allowed.

9 Phase 4: Google Chat Validation

Google Chat is optional. Use it for a short summary, not the full report.

9.1 Create a webhook

In Google Chat:

1. Open the target Chat space in a browser.
2. Click the space name.
3. Open Apps and integrations.
4. Add an incoming webhook.

5. Copy the webhook URL.

If the button is unavailable, your Google Workspace admin may have disabled incoming webhooks.

9.2 Test the webhook

```
export CHAT_WEBHOOK="https://chat.googleapis.com/v1/spaces/..."

curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"text": "Jira bug reporter test from Linux VM"}' \
  "$CHAT_WEBHOOK"
```

Checkpoint: Google Chat is nice to have. The core tool should still work even if Chat webhooks are blocked.

10 Phase 5: Create the Project Skeleton

10.1 Create folders

```
mkdir -p jira-bug-reporter/{reporter,config,output,logs,tests}
cd jira-bug-reporter
python3 -m venv .venv
. .venv/bin/activate
```

For version 1, you do not need to install any Python packages. Use:

- `urllib.request` for Jira HTTP calls.
- `json` for parsing.
- `smtplib` and `email.message` for email.
- `sqlite3` only if you enable history.
- `unittest` for tests.

10.2 Recommended folder structure

```
jira-bug-reporter/
reporter/
  __init__.py
  main.py
  config.py
  jira_client.py
  models.py
  categorizer.py
  analytics.py
  renderer.py
  email_sender.py
  chat_sender.py
  storage.py
config/
  config.example.ini
output/
logs/
tests/
```

```
test_categorizer.py
test_analytics.py
sample_jira_response.json
```

11 Phase 6: Configuration Design

Keep secrets out of the config file. Put only non-secret settings in `config.ini`.

11.1 Example config

```
[jira]
base_url = https://celestica.atlassian.net
project_key = RBC
jql = project = RBC AND issuetype in (Bug, Defect) ORDER BY priority ASC, created ASC
fields = key,summary,status,priority,assignee,reporter,created,updated

[report]
title = CruiseBay RBC Daily Bug Report
owner_name = Automation Infra
owner_email = your.email@celestica.com
new_bug_days = 2
warn_age_days = 7
critical_age_days = 15

[categories]
feedback_statuses = Feedback, Feedback Needed
test_statuses = Test, Verify Fix, QA
fixed_statuses = Fixed, Done, Resolved

[email]
enabled = true
smtp_host = smtp-relay.gmail.com
smtp_port = 587
from_address = bug-report@celestica.com
to_addresses = team1@celestica.com, team2@celestica.com
cc_addresses =

[chat]
enabled = false
```

11.2 Secrets through environment variables

```
export JIRA_EMAIL="your.email@celestica.com"
export JIRA_API_TOKEN="your_api_token"
export CHAT_WEBHOOK_URL="https://chat.googleapis.com/v1/spaces/..."
```

Watch out: Never store `JIRA_API_TOKEN` or `CHAT_WEBHOOK_URL` in Git. Treat both like passwords.

12 Phase 7: Jira Client Design

12.1 Jira API behavior

The Jira client should:

- Build the endpoint URL: `/rest/api/3/search/jql`.
- Send basic auth using email and API token.
- Request only fields needed by the report.
- Handle pagination if there are more issues than one page.
- Fail clearly on 401, 403, 429, and 5xx responses.

12.2 Pagination model

The newer JQL endpoint may return a token-based pagination style. Design the client so it can loop until there are no more pages.

```
issues = []
next_page_token = None

while True:
    response = search_jira(jql, fields, max_results=100, next_page_token=next_page_token)
    issues.extend(response.get("issues", []))
    next_page_token = response.get("nextPageToken")
    if not next_page_token:
        break
```

12.3 Data you should normalize

For each Jira issue, normalize into:

```
key
url
summary
priority
assignee
reporter
status
created
updated
days_open
reason
```

If a field is missing, use a safe default like `-` instead of crashing.

Checkpoint: After implementing the Jira client, run a test using saved sample JSON before calling the real API repeatedly.

13 Phase 8: Categorization Rules

13.1 Core sections

Section	Rule
All Bugs	Every issue returned by the JQL query.
Feedback Needed Bugs	Status is in configured feedback statuses.
Test / Verify Fix Bugs	Status is in configured test statuses.

Newly Created Bugs

Created date is within last N days, usually 2.

13.2 Age calculation

Use the created date:

```
days_open = (today_date - created_date).days
```

Do not use updated date for days open. Updated date changes when comments or fields change.

13.3 Age color

Age	Color	Meaning
0 to 7 days	Normal	Fresh or acceptable.
8 to 14 days	Yellow	Needs attention.
15+ days	Red	Aging bug. Escalate or review.

Checkpoint: Unit test the categorizer with fake bugs before building the email template.

14 Phase 9: Analytics Rules

14.1 Required metrics

Compute:

- Total all bugs.
- Total feedback needed.
- Total test / verify fix.
- Total newly created last 2 days.
- Count by status.
- Count by priority.
- Count by assignee.
- Oldest bug age.
- Count of bugs older than warning threshold.

14.2 Email-safe chart strategy

For version 1, avoid PNG charts. Many email clients block images or render them poorly. Use HTML bars:

```
Status Breakdown
Fixed [##### ] 5
Test [##### ] 3
Feedback [##### ] 3
Open [## ] 1
```

Later, if the team insists on pie charts, generate PNG attachments with `matplotlib`. Treat that as version 2 because it adds another dependency.

15 Phase 10: HTML Email Design

15.1 Email rendering rules

Use:

- HTML tables.
- Inline CSS.
- Absolute Jira links.
- Short columns.
- Escaped text using Python's `html.escape`.

Avoid:

- JavaScript.
- External CSS.
- External fonts.
- Complex flex/grid layouts.
- Huge embedded images.

15.2 Recommended email layout

```
Subject: CruiseBay RBC Daily Bug Report - 2026-05-27

Hi All,

Please find the CruiseBay RBC Daily Bug Report details below.

Overall Summary
  Feedback needed: 3
  Test / verify fix: 3
  New last 2 days: 0
  Total all bugs: 13

By Status
By Priority

All Bugs
Feedback Needed Bugs
Test / Verify Fix Bugs
Newly Created Bugs

Note: Auto-generated email. Contact <owner> for queries.

Thanks,
Automation Infra
Celestica SONiC Team
```

15.3 Columns to include

Column	Notes
BugId	Link to Jira issue.
Summary	Keep text escaped and allow wrapping.
Priority	P1/P2.
Assignee	Display name or -.
Reporter	Display name or -.
Status	Exact Jira status.
Days Open	Colored based on age threshold.
Created	Date only.
Updated	Date only.
Reason	Optional custom field if available.

Checkpoint: Open `output/latest_report.html` in a browser before sending it to the team.

16 Phase 11: Email Sender Design

16.1 SMTP configuration

The sender should support:

- SMTP host.
- SMTP port.
- STARTTLS on/off.
- Optional username/password if approved.
- To and Cc recipients.
- HTML body.

16.2 Python standard library modules

Use:

- `smtplib`
- `email.message.EmailMessage`
- `ssl`

16.3 Failure handling

If email sending fails:

- Save the HTML report anyway.
- Write the exception to logs.

- Return non-zero exit code.
- Optionally send a Chat message saying email failed.

Checkpoint: Send the first test email only to yourself. Then add the team list after layout and data are verified.

17 Phase 12: Google Chat Sender Design

The Chat sender should post a short message:

```
CruiseBay RBC Daily Bug Report
Total: 13 | P1: 8 | P2: 5 | Feedback: 3 | Test: 3
HTML report emailed to the team.
```

Keep it short. Google Chat should be the notification, not the report body.

18 Phase 13: Testing Plan

18.1 Beginner-friendly rule

Before each phase, run the tests from previous phases. This prevents you from breaking working logic while adding the next part.

18.2 Test order

Phase	Test before moving on
Config	Load config and verify required keys exist.
Jira Client	Parse sample Jira JSON into normalized issues.
Categorizer	Fake issues are assigned to correct sections.
Analytics	Counts by status and priority are correct.
Renderer	HTML file is generated and contains issue links.
Email Sender	Test email sends to yourself.
Chat Sender	Test message appears in Chat space.
Main Job	Dry run completes without sending email.
Scheduler	Timer runs and writes logs.

18.3 Useful commands

```
python3 -m unittest discover -s tests
python3 -m reporter.main --config config/config.ini --dry-run
python3 -m reporter.main --config config/config.ini --send-test-email your.email@celestica.com
```

19 Phase 14: Logging and Output

19.1 Recommended output files

```
output/latest_report.html
output/report_2026-05-27.html
logs/jira_bug_reporter.log
logs/jira_bug_reporter.err
```

19.2 Log what matters

Log:

- Job start/end time.
- JQL used.
- Number of issues fetched.
- Number of issues per category.
- Email recipients count.
- Whether email succeeded.
- Whether Chat succeeded.
- Errors with stack traces.

Do not log:

- API tokens.
- Chat webhook URLs.
- SMTP passwords.

20 Phase 15: Deployment With Systemd

Systemd is cleaner than cron for a production VM because it has logs, environment files, restart behavior, and explicit service state.

20.1 Create install directory

```
sudo mkdir -p /opt/jira-bug-reporter
sudo mkdir -p /etc/jira-bug-reporter
sudo mkdir -p /var/log/jira-bug-reporter
```

20.2 Environment file

Create:

```
/etc/jira-bug-reporter/env
```

Content:

```
JIRA_EMAIL=your.email@celestica.com
JIRA_API_TOKEN=replace_with_real_token
CHAT_WEBHOOK_URL=
```

Protect it:

```
sudo chown root:root /etc/jira-bug-reporter/env
sudo chmod 600 /etc/jira-bug-reporter/env
```

20.3 Service file

Create:

```
/etc/systemd/system/jira-bug-reporter.service
```

Content:

```
[Unit]
Description=Jira Daily Bug Reporter
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
WorkingDirectory=/opt/jira-bug-reporter
EnvironmentFile=/etc/jira-bug-reporter/env
ExecStart=/opt/jira-bug-reporter/.venv/bin/python -m reporter.main --config /opt/jira-bug-reporter/config
/config.ini
StandardOutput=append:/var/log/jira-bug-reporter/service.log
StandardError=append:/var/log/jira-bug-reporter/service.err
```

20.4 Timer file

Create:

```
/etc/systemd/system/jira-bug-reporter.timer
```

Content:

```
[Unit]
Description=Run Jira Daily Bug Reporter every weekday

[Timer]
OnCalendar=Mon..Fri 09:00
Persistent=true
Unit=jira-bug-reporter.service

[Install]
WantedBy=timers.target
```

20.5 Enable and test

```
sudo systemctl daemon-reload
sudo systemctl start jira-bug-reporter.service
sudo systemctl status jira-bug-reporter.service

sudo systemctl enable --now jira-bug-reporter.timer
systemctl list-timers | grep jira-bug-reporter
```

Checkpoint: Do one manual `systemctl start` before enabling the timer. This proves the production environment file and working directory are correct.

21 Phase 16: Security Checklist

- Use a service account if company policy allows.
- Give the Jira account only required project permissions.
- Store token in environment file with `chmod 600`.
- Never log secrets.
- Never commit `config.ini` if it contains recipient lists considered internal.
- Rotate token if someone leaves the team or the token is exposed.
- Keep webhook URL private.
- Restrict VM access.
- Use company-approved SMTP route.

22 Phase 17: Troubleshooting

Problem	Likely fix
Jira curl works locally but systemd fails	Systemd does not have your shell environment. Put variables in <code>/etc/jira-bug-reporter/env</code> .
Jira returns zero bugs	JQL is too narrow, project key is wrong, or account lacks permission.
Jira returns 401	Token/email mismatch or token revoked.
Jira returns 403	Account lacks project permission.
Jira returns 429	Too many requests. Add backoff and reduce schedule frequency.
Email sends to self but not group	SMTP relay or group permission issue. Ask Workspace admin.
Email HTML looks broken	Simplify layout, use tables, inline CSS, and avoid unsupported CSS.
Chat webhook fails	Webhook disabled, URL leaked/revoked, or network blocks Google Chat endpoint.
Timer never runs	Check <code>systemctl list-timers</code> and server timezone.

23 Implementation Milestones

23.1 Milestone 1: API proof

Done when:

- VM can call Jira.
- API token works.
- JQL returns real issues.
- Project key and fields are confirmed.

23.2 Milestone 2: Local HTML report

Done when:

- Script fetches Jira issues.
- Script generates `output/latest_report.html`.
- Tables match the expected report sections.
- Age colors work.
- Unit tests pass.

23.3 Milestone 3: Email report

Done when:

- Test email sends to yourself.
- HTML renders well in Gmail.
- Team recipients are added after approval.
- Failure logs are clear.

23.4 Milestone 4: Production schedule

Done when:

- `systemd` service runs manually.
- `systemd` timer is enabled.
- Logs are written.
- The report arrives at the expected time.

24 Version Roadmap

24.1 Version 1: Minimum useful production tool

- Jira fetch.
- HTML report.
- Email send.
- Local logs.
- Dry-run mode.
- Unit tests.

24.2 Version 2: Better reporting

- SQLite history.
- Trend: bugs opened/closed per day.
- Google Chat summary.

- Optional PNG charts if approved.
- More robust retry/backoff.

24.3 Version 3: Team-scale tool

- Multiple projects.
- Per-project recipient lists.
- Admin config file validation.
- Dashboard only if the team asks for one.
- Containerization only if deployment requires it.

25 Recommended First Week Plan

Day	Work
Day 1	Validate Jira API, project key, JQL, field names, SMTP route, and optional Chat webhook.
Day 2	Build config loader, Jira client, sample JSON, and normalizer.
Day 3	Build categorizer and analytics with unit tests.
Day 4	Build HTML renderer and compare with desired email layout.
Day 5	Add email sender, dry-run mode, logging, and self-test email.
Day 6	Deploy on Linux VM with systemd service and timer.
Day 7	Monitor first scheduled run, fix formatting, document handoff steps.

26 Definition of Done

The project is production-ready for your team when:

- A non-developer can open the report email and understand current bug health.
- The report runs automatically on schedule.
- The script survives empty result sets.
- Jira/API/email failures are logged clearly.
- Secrets are not in code.
- The team knows who owns the automation.
- There is a short README explaining setup, testing, and recovery.

27 Appendix A: Minimal Dry-run Command Set

Use this as your first real validation sequence:

```
curl -I https://celestica.atlassian.net

export JIRA_BASE="https://celestica.atlassian.net"
export JIRA_EMAIL="your.email@celestica.com"
read -s JIRA_API_TOKEN
export JIRA_API_TOKEN

curl -sS \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  -H "Accept: application/json" \
  "$JIRA_BASE/rest/api/3/myself" | python3 -m json.tool

export PROJECT_KEY="RBC"

curl -sS -G \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  -H "Accept: application/json" \
  --data-urlencode "jql=project = $PROJECT_KEY ORDER BY created DESC" \
  --data-urlencode "maxResults=5" \
  --data-urlencode "fields=key,summary,status,priority,assignee,reporter,created,updated" \
  "$JIRA_BASE/rest/api/3/search/jql" | python3 -m json.tool

nc -vz smtp-relay.gmail.com 587
```

28 Appendix B: IT Approval Checklist

Send this to IT/admin if anything is blocked:

We are building an internal scheduled Jira bug report tool on a Linux VM.

Required outbound access:

1. HTTPS to <https://celestica.atlassian.net>
2. SMTP to the approved company mail relay
3. Optional HTTPS to Google Chat webhook endpoint

Questions:

1. Can the VM use `smtp-relay.gmail.com:587`, or should it use an internal SMTP relay?
2. Should the VM public IP be allowlisted for relay?
3. What sender address should automation use?
4. Are there group restrictions for sending to team aliases?
5. Is a Jira service account available for this report?
6. Are Google Chat incoming webhooks allowed in our Workspace?

29 Appendix C: What To Build First

Build in this exact order:

1. Jira curl test.
2. SMTP curl/nc test.
3. Project skeleton.
4. Config loader.
5. Sample JSON file.
6. Normalizer.

7. Categorizer tests.
8. Analytics tests.
9. HTML renderer.
10. Dry run.
11. Email to yourself.
12. systemd deployment.

This order keeps you from getting stuck on fancy output before proving the corporate network path.